Volume xx (200y), Number z, pp. 1–5

Hybrid LOD Rendering between Mesh and Gaussian Splat Representations

Jerneja Krajcar

University of Ljubljana

Abstract

This seminar explores a hybrid level of detail (LOD) setup where a simplified mesh is used for distant views and a higher-quality Gaussian splat representation is used closer to the camera. The practical goal is not to make a perfect renderer, but to test which transition techniques reduce visible popping and preserve image quality. The prototype uses Mesh2Splat PLY files as Gaussian input, builds nested Gaussian detail levels from one selected cloud, drives the transition from camera distance, and exposes the modes in an interactive web viewer.

Keywords: 3D Gaussian Splatting, mesh rendering, level of detail, hybrid rendering, smooth transitions

1. Introduction

Meshes are compact, easy to render, and still useful when an object is far from the camera. 3D Gaussian Splatting (3DGS), as introduced by Kerbl et al. [KKLD23], can represent richer appearance with many semi-transparent primitives. This makes the task a hybrid rendering problem: use a cheaper mesh representation when high detail is not needed, and gradually move toward a Gaussian representation when the camera gets closer.

The difficult part is the transition. If the viewer simply switches from mesh to Gaussians at one distance, the object can pop, change brightness, or briefly look incomplete. The work in this prototype is therefore focused on transition techniques: distance-based blending, continuous Gaussian detail levels, depth-aware compositing, and a few different ways of deciding when the mesh should disappear.

2. Problem

The required system should explore a smooth transition between two scene representations:

- a simplified mesh, suitable for low-detail or far-away views;
- one or more Gaussian splat representations, suitable for higher-detail close-up views.

A naive switch at a fixed distance produces visible popping because the two representations do not necessarily match in color, coverage, silhouette, or sampling density. A better solution should make the change gradual and should keep neighbouring Gaussian levels spatially related, so that a denser level does not look like a completely unrelated point set.

3. Related Work

3DGS provides the basic representation used for the Gaussian side of this project [KKLD23]. SuGaR aligns Gaussians with surfaces and uses this relationship for mesh extraction and high-quality mesh rendering [GL24]. This is directly relevant because it shows that mesh and Gaussian representations can be connected through surface alignment rather than treated as unrelated assets.

SplattingAvatar embeds Gaussian splats on a mesh for real-time human avatar rendering [SWL*24]. Its main target is animatable humans, but the idea of attaching or aligning Gaussian appearance to mesh structure is related to this seminar work.

Mesh2Splat [Ele25] is used as the main practical conversion path because it offers a direct way to create Gaussian splat PLY files from mesh assets. In this workflow, the input is an existing textured mesh, and Mesh2Splat produces a splat cloud that can be loaded by the viewer as the high-detail representation. This is useful for the project because the transition can be tested from an ordinary mesh asset without first building a full custom 3DGS training pipeline. The repository also includes an experimental NVIDIA/CUDA `gsplat` training path, but this is kept separate from the Mesh2Splat workflow.

4. Implemented Solution

The prototype is centered on an interactive web viewer and a Mesh2Splat workflow. A source mesh is loaded in the viewer and one selected Gaussian PLY file is used as the high-detail Gaussian representation. The same interface can also load PLY files from the experimental trained-Gaussian path, but Mesh2Splat was the main practical source during testing. The project implementation is available in the repository `https://github.com/jernejakrajcar/meshtogaussian`.

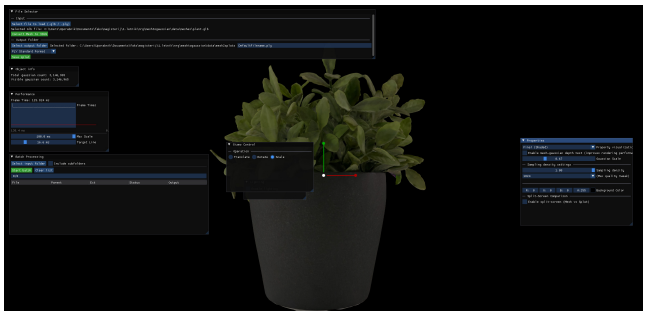

Figure 1: *Mesh2Splat result loaded in the actual viewer. The source plant mesh is used together with the generated Gaussian PLY, so transition modes can compare the simplified mesh and the splat representation in the same interface.*

In the current workflow, the viewer does not require a manually exported PLY file for every detail level. Instead, it builds a deterministic nested sequence from the selected Gaussian cloud. Smaller versions are prefixes of the same ordering, and denser versions contain all splats from coarser ones. This keeps neighbouring levels spatially related and avoids a common popping artifact where two levels look like unrelated point sets.

The viewer contains several transition modes. They are not all equally successful; some are intentionally kept as baselines because they show what goes wrong. The most useful current modes are **Detail over mesh (optimized)**, **Coverage-scaled detail over mesh**, and **Keep mesh + add detail**. The simpler cross-fade, dense cutover, and non-optimized detail modes are useful for comparison.

4.1. Transition Model

All transition modes are controlled by camera distance. The viewer maps the current camera radius to a normalized transition parameter

$$t = \mathrm{clamp}\left(\frac{r_{\mathrm{far}} - r}{r_{\mathrm{far}} - r_{\mathrm{near}}}, 0, 1\right),$$

where r is the distance from the camera to the orbit target. In the default configuration, $r_{\mathrm{far}} = 4.0$ and $r_{\mathrm{near}} = 1.25$ in normalized object units. When the viewer slider is used, it also moves the camera along this far-to-near range, so the slider and camera-distance transition use the same parameter.

For detail-style transitions, t is converted into a target number of active splats. This acts like a continuous detail factor:

$$N_{\mathrm{active}} = f(t)\, N_{\mathrm{total}},$$

The function $f(t)$ is deliberately not a hard step. The viewer uses smoothstep ramps so that the number of visible splats changes gradually as the camera moves. This is important because the transition is distance-based: when the camera is far away, the mesh can carry most of the image; when the camera moves closer, more Gaussian detail is introduced. The modes differ mainly in how quickly they reveal splats and how long they keep the mesh visible.

4.2. Gaussian Detail Sampling

The Gaussian sequence is not sampled randomly. The selected Gaussian cloud is sorted once by a deterministic importance and spatial-coverage heuristic. Importance is based on opacity and scale, while spatial distribution is encouraged by assigning candidates to a 3D grid and taking splats from occupied regions in a round-robin style. The intention is that a small prefix still covers different parts of the object instead of selecting only strong splats from one local area.

The optimized transition modes use this nested order directly. If the transition asks for N active splats, the viewer chooses the smallest prepared level that contains at least N splats, then slices the first N entries from that level. The count is rounded up to a small bucket size to avoid rebuilding WebGL buffers for every tiny camera movement. This is why the log may show a target such as 307,760 splats but a drawn subset such as 309,248 splats.

This became necessary after the first detail mode. That mode could reveal splats continuously, but it still uploaded and drew the full dense cloud while hiding inactive splats in the shader. For a large asset such as `plant-3146965.ply`, this is too expensive for interactive testing. The optimized path instead uploads and draws only the active subset, which keeps the continuous transition idea but makes the viewer more responsive.

4.3. Mesh and Gaussian Alignment

The mesh and Gaussian data must be in the same coordinate system before a transition can look plausible. The source mesh is normalized to a unit sphere for viewer display. For Mesh2Splat PLY files exported from the same source asset, the Gaussian positions and scales are normalized with the same mesh centre and radius. This keeps the two representations in the same object space.

For trained Gaussian files, the code also includes an alignment heuristic. It estimates robust bounds for the mesh and Gaussian cloud, tries signed axis permutations, and chooses the transform with the lowest average nearest-neighbour distance to the mesh sample. Gaussian covariance axes are transformed together with position, so anisotropic splats remain consistently oriented after alignment.

4.4. Depth-Aware Compositing

The viewer renders the mesh with Three.js and Gaussian splats with a custom raw WebGL2 splat renderer. This is the main Gaussian display path in the interactive viewer: splat attributes are uploaded to GPU buffers and rendered as instanced screen-space quads. The UI also includes a small GPU rasterizer debug toggle, which does not change the render path, but exposes whether the browser is currently using the WebGL2 path and what rasterizer limits are active.

Without additional handling, rear-side splats can be alpha-blended over a visible front mesh surface because the mesh and

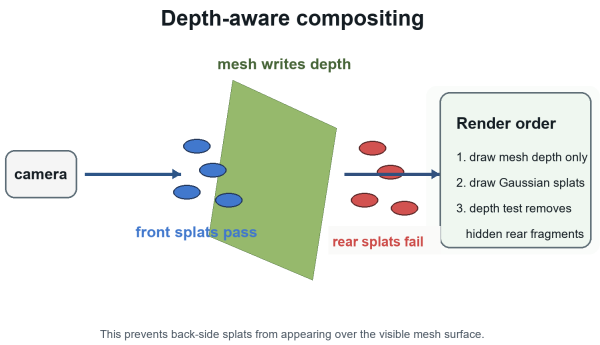

Figure 2: *Depth-aware compositing. The mesh writes only depth into the Gaussian renderer, then rear splat fragments are rejected by the depth test.*

Gaussian layers do not share a normal depth buffer. This caused visible discontinuities during hybrid transitions.

The current detail-style modes therefore use a depth-aware mask. Before drawing Gaussian splats, the Gaussian renderer draws the mesh geometry invisibly into its own depth buffer. Splats are then rendered with depth testing enabled. Fragments that are behind the front mesh surface fail the depth test, while front/surface splats remain visible. A small polygon offset is used when writing mesh depth so that surface splats are not removed by numerical precision issues.

4.5. Transition Modes

Cross-fade mesh to splats changes opacity between the mesh and Gaussian layers. This is the easiest transition to implement, but it is also the least convincing visually. For a part of the transition both representations are visible at once, which can create double surfaces and a transparent-looking object. It is useful as a baseline, but not as a final solution.

Detail over mesh keeps the mesh visible, then reveals splats from the dense Gaussian cloud. This reduced popping compared to a hard switch because the mesh still provides the object silhouette while the Gaussian representation appears. The problem is performance: the first version still uploaded and drew the full dense cloud, even when only part of it was visible.

Detail over mesh (optimized) keeps the same idea but uses the nested Gaussian order. The viewer renders an increasing prefix of the Gaussian cloud and only uploads the active subset. This makes the transition much more practical for large PLY files. It also shows a limitation: when the mesh fades out before the active subset has enough coverage, holes can appear. On the plant model this is visible around 96% transition, where most splats are already active, but some thin leaves are still not covered well enough.

Coverage-scaled detail over mesh is a small modification of the optimized mode. It starts adding splats earlier and temporarily increases Gaussian scale while the subset is sparse. This is not physically perfect, but it helps with coverage. For the plant test case this mode looked more stable than plain optimized detail, because a small amount of blur was less distracting than empty regions on the leaves.

Keep mesh + add detail does not try to replace the mesh. The mesh stays visible for the whole transition and Gaussian splats are added as extra detail. This was visually good for assets where the original mesh texture is sharper than the Gaussian cloud. The trade-off is conceptual: it is no longer a full mesh-to-Gaussian handoff, but a hybrid rendering mode. Depth-aware compositing is important here, otherwise rear splats are drawn over the front mesh surface.

Dense cutover fades directly to the densest available Gaussian level. It is a useful reference for the final Gaussian quality, but it gives less control over the middle of the transition and can still feel abrupt.

5. Alternatives Considered

Hard level switching would be simpler, but it directly causes popping. It is not enough for a smooth transition task.

Opacity-only cross-fading was implemented first because it is easy to inspect, but it makes the transition look like a transparency effect rather than a real representation change.

Shader-only reveal masking was useful for early experiments because it allowed continuous reveal from a dense cloud. It was later superseded by optimized subset rendering, which avoids drawing inactive splats.

Depth-aware screen-space blending would go further than the current depth mask by blending final images using per-pixel color and depth from both renderers. The implemented depth mask solves the most visible rear-splat-over-mesh artifact, but it is not yet a full image-space composition algorithm.

Direct CUDA/gsplat training was added as an experimental route for generating splats from synthetic training views on an NVIDIA/CUDA machine. It was not kept as the main path because it depends on a heavier CUDA, PyTorch, compiler, and training setup.

Full surface-aligned Gaussian optimization in the style of SuGaR would likely improve geometric consistency, but it is heavier than the current seminar prototype and would require a stronger training and reconstruction pipeline.

6. Results and Discussion

The prototype currently focuses on the interactive viewer. It loads a source mesh together with a selected Gaussian PLY file and displays continuous transition modes controlled by camera distance.

The main result is that there is no single transition mode that is best in every case. **Detail over mesh (optimized)** is the cleanest conceptual mesh-to-Gaussian transition because the mesh eventually disappears and the Gaussian representation becomes the final view. However, Figure 3 shows that it can expose sparse coverage on complex foliage before the final dense cloud is fully active. **Coverage-scaled detail over mesh** handled that plant case better,

Detail over mesh (optimized) | Coverage-scaled detail | Keep mesh + add detail

Figure 3: *Comparison of three plant transition modes at 96% transition in the viewer.* ***Detail over mesh (optimized)*** *is the cleanest handoff idea, but can still show uneven coverage on thin leaves.* ***Coverage-scaled detail*** *fills those gaps more reliably by temporarily increasing splat size, at the cost of some softness.* ***Keep mesh + add detail*** *gives the most stable silhouette because the mesh remains visible, but it is a hybrid detail mode rather than a full replacement by Gaussians.*

because it hides coverage gaps during the transition. **Keep mesh + add detail** was also visually strong, especially when the mesh texture was sharper than the Gaussian PLY, but it should be understood as a hybrid view rather than a full replacement of the mesh.

This is also why depth-aware compositing mattered. Without it, keeping the mesh visible while adding Gaussian detail looked wrong because rear splats could appear over the front surface. After adding the mesh depth pass, the hybrid modes became more useful: the mesh could carry the silhouette and texture while the Gaussian layer added detail only where it should be visible.

From the scalability point of view, the continuous subset was the most important optimization. Drawing the full dense cloud and hiding inactive splats is simple, but it does not scale to multi-million-splat examples. Drawing only the active nested prefix keeps the number of rendered splats tied to camera distance. This is the main idea behind the hybrid approach: spend less work when the camera is far away, and only pay for dense Gaussian detail when it is actually visible.

In a larger renderer this kind of hybrid approach could improve scalability while keeping high image quality across levels of detail. A mesh is still a good coarse representation: it gives a stable silhouette, cheap rendering, and reliable texture when the object is far away. Gaussians are useful when close-up appearance matters. The transition between them is the hard part, and this prototype suggests that it needs both continuous detail selection and some form of depth-aware composition.

6.1. Live FPS Display

Instead of treating FPS as a fixed benchmark number, the viewer shows a small live performance overlay in the top-right corner. This is more honest for this prototype because the result depends on the local GPU, browser, viewport size, camera position, active transition mode, and number of currently drawn splats. A model such as `plant-3146965.ply` contains about 3.15M splats, so the absolute FPS value is expected to be heavy and should not be read as a general renderer score.

The overlay shows the current FPS, average frame time, visible splat count, active mode, transition percentage, WebGL2 status, and viewport size. For a rough average FPS comparison, the same display can be observed while moving the transition slider from 0% to 100% with the camera locked. This gives a performed-live measurement for the current machine and setup, which is useful for comparing modes during development, but not precise enough to present as a formal evaluation table.

The HUD calculation is intentionally simple:

```
function updatePerformanceHud(now) {
  frameTimes.push(now - lastFrameAt);
  if (frameTimes.length > 90) frameTimes.shift();
  lastFrameAt = now;

  const avgFrameMs =
    frameTimes.reduce((sum, value) => sum + value,
        0) /
    frameTimes.length;
  const fps = 1000 / avgFrameMs;
  showHud(fps, avgFrameMs,
      currentGaussianDrawCount());
}
```

The CPU/offline renderer remains useful for validation, but it is no longer presented as the main route for high-quality Gaussian generation. The added NVIDIA/CUDA training path can produce trained Gaussian PLY files for the viewer, and the viewer can display them through the default raw WebGL2 Gaussian rasterizer. The added GPU debug option is mainly a practical check that this browser-side path is active during experiments.

7. Limitations and Future Work

The largest current limitation is Gaussian close-up quality. In the tested textured plant model, the original mesh can look sharper than the Gaussian representation because the mesh displays the original texture directly, while the Gaussian cloud approximates appearance with a finite set of colored splats. This does not mean the hybrid idea is wrong, but it shows that the quality of the Gaussian input strongly affects the result.

The current WebGL renderer is also a simplified splat renderer, not a production 3DGS rasterizer. It is good enough for comparing transition behaviour in the viewer, but it can still show blur, stripe-like alpha blending, and imperfect sorting artifacts. The GPU debug toggle and FPS HUD help inspect the browser path, but they do not solve these rendering limitations.

The practical workflow still depends on externally exported

Mesh2Splat PLY files. The CUDA/`gsplat` route adds dataset export and trainer execution support, but it is experimental and depends on an external NVIDIA training environment. A more complete version should compare several assets with measured FPS and image-quality metrics, not only visual inspection.

The main improvements would be: better Gaussian export or training quality, a more production-like Gaussian rasterizer, better surface alignment between mesh and splats, and automatic tuning of transition curves per asset. Surface-aligned methods such as SuGaR suggest one possible direction, because they treat the mesh and Gaussian representation as related rather than completely separate.

8. Conclusion

The prototype explores several techniques for transitioning between a simplified mesh and a higher-quality Gaussian representation. The most useful parts were camera-distance control, deterministic nested Gaussian levels, optimized subset rendering, and depth-aware mesh/Gaussian compositing.

The result is not a seamless final renderer, but it shows which pieces are needed. A hard switch is too visible. A simple opacity fade is easy but often looks like transparency. Keeping the mesh visible for longer improves stability, especially when combined with depth-aware splat rendering. For assets with sparse Gaussian coverage, coverage-scaled detail can look better than a pure fade-out of the mesh. Overall, the hybrid approach can improve scalability because dense Gaussian detail is only used where it matters, while the mesh still provides a cheap and stable representation at lower levels of detail.

References

[Ele25] ELECTRONIC ARTS: Mesh2Splat: Fast mesh to 3d gaussian splat conversion. https://github.com/electronicarts/mesh2splat, 2025. 1

[GL24] GUÉDON A., LEPETIT V.: SuGaR: Surface-aligned gaussian splatting for efficient 3d mesh reconstruction and high-quality mesh rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2024). URL: https://openaccess.thecvf.com/content/CVPR2024/html/Guedon_SuGaR_Surface-Aligned_Gaussian_Splatting_for_Efficient_3D_Mesh_Reconstruction_and_CVPR_2024_paper.html. 1

[KKLD23] KERBL B., KOPANAS G., LEIMKÜHLER T., DRETTAKIS G.: 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics 42*, 4 (2023). doi:10.1145/3592433. 1

[SWL*24] SHAO Z., WANG Z., LI Z., WANG D., LIN X., ZHANG Y., FAN M., WANG Z.: Splattingavatar: Realistic real-time human avatars with mesh-embedded gaussian splatting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2024). URL: https://openaccess.thecvf.com/content/CVPR2024/papers/Shao_SplattingAvatar_Realistic_Real-Time_Human_Avatars_with_Mesh-Embedded_Gaussian_Splatting_CVPR_2024_paper.pdf. 1